

上位机JSON 命令示例

1. 单个静态动作

```
1 {
2   "name": "静态姿势",
3   "version": 2,
4   "steps": [
5     {
6       "servos": [
7         { "id": 1, "position": 500, "time": 0 },
8         { "id": 2, "position": 620, "time": 0 },
9         { "id": 3, "position": 480, "time": 0 }
10      ],
11       "delay": 200
12     }
13   ]
14 }
```

- `delay` 字段**可选**，默认值为 0，解析时若不存在则按 0 处理。
- 用户若不需要延时，可以省略 `delay` 或设为 0。

2. 基础线性序列（无循环/跳转）

每个动作序列包含一个 `steps` 数组，每个 step 对应一个静态动作组（即一条 0x02 指令）。

```
1 {
2   "name": "前进摆动",
3   "version": 3,
4   "steps": [
5     {
6       "servos": [
7         { "id": 1, "position": 500, "time": 200 },
8         { "id": 2, "position": 600, "time": 200 },
9         { "id": 3, "position": 500, "time": 200 }
10      ],
11       "delay": 200
12     },
13     {
14       "servos": [
15         { "id": 1, "position": 600, "time": 300 },
16         { "id": 2, "position": 500, "time": 300 },
17         { "id": 3, "position": 600, "time": 300 }
18      ],
19       "delay": 300
20     },
21     {
22       "servos": [
23         { "id": 1, "position": 800, "time": 300 },
24         { "id": 2, "position": 400, "time": 300 },
25         { "id": 3, "position": 800, "time": 300 }
26      ],
27       "delay": 300
28     }
29   ]
30 }
```

```
28     }
29   ]
30 }
```

- `servos` 数组中每个元素为 [舵机ID, 目标位置脉冲(0~1000), 运行时间(ms)]。
- 用户自定义每个指令发送的时间间隔 `delayx`
- 没有 `control` 类型步骤, 默认动作序列只执行一次

3. 带控制指令的序列（支持循环、等待、跳转）

若需在JSON内直接表达循环逻辑（例如重复执行某几个步骤），可增加 `control` 类型步骤。

```
1  {
2    "name": "前进摆动",
3    "version": 3,
4    "steps": [
5      {
6        "servos": [
7          { "id": 1, "position": 500, "time": 200 },
8          { "id": 2, "position": 600, "time": 200 },
9          { "id": 3, "position": 500, "time": 200 }
10       ],
11       "delay": 200
12     },
13     {
14       "servos": [
15         { "id": 1, "position": 600, "time": 300 },
16         { "id": 2, "position": 500, "time": 300 },
17         { "id": 3, "position": 600, "time": 300 }
18       ],
19       "delay": 300
20     },
21     {
22       "servos": [
23         { "id": 1, "position": 800, "time": 300 },
24         { "id": 2, "position": 400, "time": 300 },
25         { "id": 3, "position": 800, "time": 300 }
26       ],
27       "delay": 300
28     }
29   ],
30   "playback": {
31     "loop": {
32       "enabled": true,
33       "count": 5,
34       "startIndex": 0,
35       "endIndex": 1,
36       "delayBetweenLoops": 500
37     }
38   }
39 }
```

字段说明

字段	类型	默认值	说明
<code>playback.loop.enabled</code>	boolean	<code>false</code>	是否启用循环
<code>playback.loop.count</code>	integer	1	循环次数： <code>-1</code> 表示无限循环；正整数表示有限次数（包括第一次）。例如 <code>count=3</code> 表示循环区间执行 3 次。
<code>playback.loop.startIndex</code>	integer	0	循环起始 step 索引（0-based），包含。
<code>playback.loop.endIndex</code>	integer	最后一帧	循环结束 step 索引（包含）。
<code>playback.loop.delayBetweenLoops</code>	integer	0	每轮循环结束后（即执行完 <code>endIndex</code> 对应的 step 后）额外等待的毫秒数，再开始下一轮循环。

4.关于嵌套循环

如果你有一系列的动作组，动作组之间有 某一段需要循环，上面的方式二已经可以满足需求

所谓的嵌套循环就是，你进入了 某几个循环的动作组

但是 在循环内部，你想继续 再循环某几个动作 线性执行...

算了，手动展开吧，也就是多复制粘贴几次的事

因为内部循环不可能是无限循环。